

내 상황에 맞는 머신러닝 설계 가이드북

지도학습과 비지도학습 선택부터 전처리, 로그 변환, 표준화, 하이퍼파라미터 튜닝, 평가 지표 해석까지 한 번에 연결하는 실무형 의사결정 가이드입니다.

문제 정의

무엇을 예측하거나 발견할지
먼저 정합니다.

설계 선택

상황에 맞는 학습 유형과 모델
계열을 고릅니다.

평가와 개선

숫자를 읽고 다음 행동으로 연
결합니다.

상세 목차

1. 이 가이드북을 읽는 방법
2. 머신러닝을 써야 하는 상황인가?
3. 지도학습, 비지도학습, 강화학습 선택
4. 지도학습 선택: 회귀, 분류, 시계열, 추천
5. 비지도학습 선택: 군집, 차원축소, 이상탐지
6. 데이터 이해와 문제 정의
7. 데이터 전처리 기본기
8. 로그 변환과 데이터 변환
9. 표준화와 정규화
- .0. 모델 선택과 기준 모델
 - .1. 학습, 검증, 테스트 분리
 - .2. 하이퍼파라미터 튜닝
 - .3. 예측 후 평가와 결과 해석
 - .4. 오류 분석과 성능 개선 순서
 - .5. 모델 해석, 배포, 운영
 - .6. 실무 체크리스트와 예시 시나리오
 - .7. 보강 노트: 놓치기 쉬운 실전 포인트

1. 이 가이드북을 읽는 방법

이 문서는 머신러닝 개념을 나열하는 사전이 아니라, “내가 가진 문제에서 어떤 선택을 해야 하는가?”를 판단하기 위한 설계 지도입니다. 실제 프로젝트에서는 모델을 고르기 전에 문제 정의, 데이터의 시점, 평가 기준을 먼저 정해야 합니다.

1단계. 머신러닝이 필요한 문제인지 확인한다.



2단계. 정답 데이터가 있는지 보고 지도학습과 비지도학습을 고른다.



3단계. 예측 대상이 숫자인지 범주인지, 시간 흐름이 중요한지 판단한다.



4단계. 데이터 전처리, 변환, 모델, 평가 지표를 문제에 맞게 연결한다.

좋은 머신러닝 설계는 “가장 복잡한 모델”을 고르는 일이 아닙니다. 내가 피하고 싶은 실수, 사용할 수 있는 데이터, 운영 환경을 기준으로 적절한 복잡도를 고르는 일입니다.

2. 머신러닝을 써야 하는 상황인가?

모든 자동화 문제가 머신러닝 문제는 아닙니다. 규칙이 명확하고 예외가 적다면 규칙 기반 시스템이 더 빠르고 안전합니다. 반대로 데이터 속 패턴이 복잡하고 사람이 규칙을 직접 쓰기 어렵다면 머신러닝이 좋은 선택이 될 수 있습니다.

상황	권장 접근	예시
규칙이 명확하고 자주 바뀌지 않음	규칙 기반	나이가 19세 미만이면 미성년자로 분류
과거 데이터와 정답이 있고 패턴을 학습해야 함	지도학습	고객 이탈 여부, 집값, 불량 여부 예측
정답은 없지만 구조나 그룹을 찾고 싶음	비지도학습	고객 세그먼트, 문서 묶기, 이상 거래 탐색
행동을 선택하고 보상을 통해 개선해야 함	강화학습	게임, 로봇 제어, 일부 최적화 문제

머신러닝을 쓰기 위험한 경우도 있습니다. 데이터가 너무 적거나, 정답 라벨이 부정확하거나, 예측 결과를 설명해야 하는 규제가 강한데 해석 방법이 없다면 먼저 문제를 다시 설계해야 합니다.

3. 지도학습, 비지도학습, 강화학습 선택

지도학습

입력 데이터와 정답이 함께 있을 때 사용합니다. 예를 들어 고객 정보와 실제 이탈 여부가 있으면 “이 고객이 이탈할까?”를 학습할 수 있습니다. 실무 머신러닝 프로젝트의 상당수는 지도학습에서 시작합니다.

비지도학습

정답 없이 데이터의 구조를 찾습니다. 고객을 비슷한 그룹으로 나누거나, 차원을 줄여 시각화하거나, 평소와 다른 데이터를 찾는 데 사용합니다.

강화학습

모델이 행동을 선택하고 보상을 받으며 학습합니다. 일반적인 표 데이터 예측보다 설계 난도가 높고, 시뮬레이션 환경이나 반복적인 피드백 구조가 필요합니다.

빠른 판단: 정답 컬럼이 있으면 지도학습, 정답 없이 묶거나 줄이고 싶으면 비지도학습, 행동의 결과로 보상이 누적되는 문제라면 강화학습을 검토합니다.

4. 지도학습 선택: 회귀, 분류, 시계열, 추천

지도학습에서는 타깃 변수의 성격을 먼저 봅니다. 예측하려는 값이 연속적인 숫자라면 회귀, 정해진 범주라면 분류입니다. 시간 순서가 핵심이면 시계열 문제로 따로 다루는 것이 안전합니다.

문제 유형	타깃	예시	대표 지표
회귀	연속 숫자	매출, 가격, 수요량, 온도	MAE , RMSE , R ²
이진 분류	두 클래스	이탈/유지, 불량/정상, 사기/정상	Precision , Recall , F1 , ROC-AUC
다중 분류	여러 클래스 중 하나	품종 분류, 고객 등급 분류	Accuracy , Macro F1
시계열 예측	시간 순서가 있는 값	월별 매출, 일별 방문자 수	MAE , RMSE , MAPE
추천·랭킹	사용자에게 보여줄 항목 순위	상품 추천, 콘텐츠 추천	Precision@K , NDCG

예시: “다음 달에 고객이 이탈할까?”는 이진 분류입니다. “다음 달 매출이 얼마일까?”는 회귀 또는 시계열 예측입니다. “고객에게 어떤 상품을 먼저 보여줄까?”는 추천·랭킹 문제입니다.

5. 비지도학습 선택: 군집, 차원축소, 이상탐지

비지도학습은 정답이 없는 상태에서 데이터의 구조를 발견합니다. 다만 결과가 반드시 “정답”을 의미하지는 않으므로, 업무적으로 해석 가능한지 확인해야 합니다.

목적	방법	질문
비슷한 대상을 묶고 싶다	군집화: K-Means, DBSCAN, Hierarchical Clustering	각 그룹을 설명할 수 있는가?
변수가 너무 많다	차원축소: PCA, t-SNE, UMAP	정보 손실을 감수할 만큼 단순화가 필요한가?
평소와 다른 데이터를 찾고 싶다	이상탐지: Isolation Forest, One-Class SVM	이상이 업무적으로 위험한 신호인가?

6. 데이터 이해와 문제 정의

모델링 전에 가장 중요한 것은 타겟, 예측 시점, 사용 가능한 데이터를 명확히 하는 것입니다. 이 단계가 흔들리면 아무리 좋은 모델을 써도 실제로 사용할 수 없는 결과가 나옵니다.

- **타겟 정의:** 무엇을 맞히려는가? 예: 30일 이내 이탈 여부.
- **예측 시점:** 언제 예측할 것인가? 예: 매월 1일 기준.
- **피쳐 범위:** 예측 시점에 실제로 알 수 있는 정보만 사용할 것.
- **성공 기준:** 어떤 지표가 어느 정도면 쓸 만한가?

데이터 누수: 미래 정보나 정답을 암시하는 컬럼이 학습에 들어가는 실수입니다. 예를 들어 이탈 예측 모델에 **해지일**, **해지처리상태**가 들어가면 성능은 좋아 보이지만 실제 예측에는 사용할 수 없습니다.

7. 데이터 전처리 기본기

전처리는 모델이 데이터를 제대로 읽을 수 있게 정리하는 과정입니다. 좋은 전처리는 모델 성능을 올릴 뿐 아니라, 평가 결과를 믿을 수 있게 만듭니다.

작업	언제 필요한가	대표 방법
결측치 처리	값이 비어 있을 때	삭제, 평균/중앙값 대체, 별도 범주 처리
이상치 처리	비정상적으로 큰 값이나 작은 값이 있을 때	확인 후 제거, Winsorizing, 로그 변환
범주형 인코딩	문자 범주를 모델 입력으로 바꿀 때	One-Hot Encoding, Ordinal Encoding, Target Encoding
날짜 처리	날짜/시간 컬럼이 있을 때	요일, 월, 분기, 공휴일, 경과일 생성
불균형 처리	한 클래스가 매우 적을 때	class weight, oversampling, undersampling, threshold 조정

전처리 기준은 반드시 학습 데이터에서만 학습하고, 검증/테스트 데이터에는 그 기준을 적용해야 합니다. 예를 들어 평균값 대체나 표준화 평균·표준편차를 전체 데이터로 계산하면 평가가 낙관적으로 왜곡될 수 있습니다.

8. 로그 변환과 데이터 변환

로그 변환은 값의 분포가 한쪽으로 심하게 치우쳐 있거나, 큰 값 몇 개가 모델을 흔들 때 사용합니다. 특히 매출, 소득, 가격, 방문자 수처럼 0 이상이고 긴 꼬리를 가진 데이터에서 자주 검토합니다.

상황	로그 변환 검토 이유	주의점
매우 큰 값이 일부 존재	큰 값의 영향력을 줄임	해석할 때 원래 단위로 되돌려야 함
오차가 값의 크기에 비례	절대 차이보다 비율 차이를 학습하기 쉬움	0 또는 음수가 있으면 <code>log1p</code> 나 다른 변환 고려
타깃 분포가 심하게 오른쪽으로 치우침	회귀 모델이 안정적으로 학습할 수 있음	평가 지표를 원래 스케일에서도 확인

실무 팁: 값에 0이 포함되어 있으면 `log(x)` 대신 `log1p(x)` 를 많이 씁니다. 이는 `log(1 + x)` 를 의미합니다.

로그 외에도 제곱근 변환, Box-Cox, Yeo-Johnson 변환이 있습니다. Box-Cox는 양수 데이터에 적합하고, Yeo-Johnson은 0이나 음수가 있어도 사용할 수 있습니다.

9. 표준화와 정규화

스케일링은 변수들의 크기 단위를 맞추는 작업입니다. 키는 170, 소득은 50000000처럼 변수 규모가 크게 다른 거리나 기울기에 민감한 모델이 특정 변수에 과도하게 끌릴 수 있습니다.

방법	의미	추천 상황
표준화	평균 0, 표준편차 1로 변환	선형 모델, 로지스틱 회귀, SVM, KNN, 신경망, PCA
정규화	값을 보통 0~1 범위로 변환	거리 기반 모델, 이미지 픽셀, 범위 제한이 유용한 경우

트리 기반 모델인 Decision Tree, Random Forest, Gradient Boosting 계열은 대체로 스케일링 영향이 작습니다. 하지만 선형 모델, SVM, KNN, 신경망, PCA에서는 표준화가 매우 중요할 수 있습니다.

10. 모델 선택과 기준 모델

처음부터 복잡한 모델을 고르기보다 단순한 기준 모델을 먼저 세우는 것이 좋습니다. 기준 모델은 “이 정도는 이겨야 머신러닝을 쓸 의미가 있다”는 출발선입니다.

상황	먼저 시도할 모델	이유
회귀 기본	Linear Regression, Random Forest, Gradient Boosting	해석성과 성능의 균형을 볼 수 있음
분류 기본	Logistic Regression, Random Forest, XGBoost/LightGBM	기준선과 강한 모델을 함께 비교 가능
해석이 중요	Linear/Logistic Regression, Decision Tree	설명하기 쉬움
표 형태 데이터에서 성능이 중요	Gradient Boosting 계열	실무 표 데이터에서 강한 경우가 많음
이미지·음성·복잡한 텍스트	신경망, 사전학습 모델	비정형 데이터 표현 학습에 강함

기준 모델 예시: 회귀에서는 항상 평균값을 예측하는 모델, 분류에서는 가장 많은 클래스로만 예측하는 모델, 시계열에서는 어제 값이 오늘도 유지된다고 보는 모델을 기준선으로 둘 수 있습니다.

11. 학습, 검증, 테스트 분리

모델 성능은 학습에 쓰지 않은 데이터에서 평가해야 합니다. 데이터를 나누지 않고 같은 데이터로 학습과 평가를 하면 실제 성능보다 훨씬 좋아 보입니다.

- **Train:** 모델이 학습하는 데이터
- **Validation:** 모델 선택과 튜닝에 사용하는 데이터
- **Test:** 마지막에 한 번 확인하는 최종 평가 데이터

시계열 데이터는 랜덤으로 섞어 나누면 미래 정보가 과거 학습에 들어갈 수 있습니다. 시간 순서를 유지하고 과거로 학습해 미래를 평가해야 합니다.

데이터가 적다면 교차검증을 사용할 수 있습니다. 교차검증은 데이터를 여러 조각으로 나누어 번갈아 검증하면서 성능 추정의 흔들림을 줄입니다.

12. 하이퍼파라미터 튜닝

모델이 데이터로부터 학습하는 값은 파라미터이고, 사람이 학습 전에 정하는 설정값은 하이퍼파라미터입니다. 예를 들어 트리의 깊이, 학습률, 이웃 개수, 정규화 강도 등이 하이퍼파라미터입니다.

방법	설명	추천 상황
Grid Search	정해진 후보 조합을 모두 탐색	후보가 적고 계산 비용이 낮을 때
Random Search	후보 범위에서 무작위로 탐색	탐색 공간이 넓을 때
Bayesian Optimization	이전 결과를 바탕으로 유망한 조합을 탐색	학습 비용이 크고 효율이 중요할 때
Early Stopping	검증 성능이 더 좋아지지 않으면 학습 중단	Boosting, 신경망 계열

튜닝은 보통 마지막 쪽 작업입니다. 문제 정의, 데이터 누수, 전처리, 피처 엔지니어링, 평가 지표가 잘못되어 있으면 하이퍼파라미터를 아무리 조정해도 근본적인 성능 개선이 어렵습니다.

13. 예측 후 평가와 결과 해석

평가는 숫자를 계산하는 것으로 끝나지 않습니다. 그 숫자가 실제 업무에서 어떤 의미인지 읽어야 합니다. 예를 들어 `RMSE=10` 은 타깃의 단위가 원인인지, 만 원인인지, 점수인지에 따라 의미가 완전히 달라집니다.

문제	주요 지표	읽는 방법
회귀	<code>MAE</code> , <code>RMSE</code> , <code>R²</code> , <code>MAPE</code>	평균 오차, 큰 오차, 설명력, 퍼센트 오차를 함께 본다.
분류	<code>Accuracy</code> , <code>Precision</code> , <code>Recall</code> , <code>F1</code>	오탐과 미탐 중 무엇이 더 위험한지 기준으로 읽는다.
불균형 분류	<code>PR-AUC</code> , <code>F1</code> , <code>Recall</code>	정확도 착시를 피하고 희소 클래스 성능을 확인한다.
추천	<code>Precision@K</code> , <code>Recall@K</code> , <code>NDCG</code>	상위 노출 결과가 실제로 쓸 만한지 본다.

좋은 평가 질문: “이 점수가 기준 모델보다 나은가?”, “실제 비용으로 환산하면 쓸 만한가?”, “특정 집단에서만 성능이 낮지 않은가?”, “운영에서 허용 가능한 속도와 비용인가?”

14. 오류 분석과 성능 개선 순서

모델이 틀린 샘플을 직접 보는 것은 매우 강력한 개선 방법입니다. 평균 점수만 보면 놓치는 패턴이 오류 샘플에서는 선명하게 보일 때가 많습니다.

- 1. 문제 정의와 타깃이 올바른지 확인한다.
- 2. 데이터 누수가 없는지 확인한다.
- 3. 평가 지표가 목적과 맞는지 확인한다.
- 4. 결측치, 이상치, 중복, 라벨 오류를 점검한다.
- 5. 오류가 큰 샘플 또는 자주 틀리는 집단을 본다.
- 6. 피처를 추가하거나 더 나은 표현으로 바꾼다.
- 7. 모델을 바꾸거나 앙상블을 검토한다.
- 8. 마지막으로 하이퍼파라미터 튜닝을 강화한다.

성능이 낮을 때 바로 복잡한 모델로 넘어가면 원인을 놓치기 쉽습니다. 실제로는 라벨 오류, 데이터 누수, 잘못된 평가 지표, 부족한 피처가 더 흔한 원인일 수 있습니다.

15. 모델 해석, 배포, 운영

모델을 실제로 쓰려면 “왜 이런 예측을 했는가?”와 “시간이 지나도 계속 잘 맞는가?”를 관리해야 합니다. 특히 사람의 의사결정에 영향을 주는 모델은 해석 가능성과 모니터링이 중요합니다.

주제	내용	도구/방법
해석	어떤 변수가 예측에 영향을 주었는지 확인	Feature Importance, Permutation Importance, SHAP
배포	모델을 저장하고 앱, API, 배치 작업으로 연결	pickle/joblib, REST API, Streamlit, Batch Pipeline
모니터링	입력 데이터와 예측 성능의 변화를 추적	성능 지표, 데이터 드리프트, 예측 분포 확인
재학습	데이터가 변하면 주기적으로 모델 갱신	월간/분기별 재학습, 성능 하락 시 재학습

16. 실무 체크리스트와 예시 시나리오

프로젝트 시작 체크리스트

- 예측하려는 타겟이 명확한가?
- 예측 시점에 실제로 알 수 있는 데이터만 사용하는가?
- 기준 모델을 정의했는가?
- 평가 지표가 업무 목적과 연결되는가?
- train, validation, test가 올바르게 분리되었는가?
- 전처리 기준을 train 데이터에서만 학습했는가?
- 결과를 실제 의사결정으로 어떻게 사용할지 정했는가?

예시 1: 고객 이탈 예측

문제 유형은 이진 분류입니다. 타겟은 “다음 30일 이내 이탈 여부”처럼 기간을 명확히 정의합니다. 이탈 고객 이 적다면 Accuracy보다 Recall, Precision, F1, PR-AUC를 봅니다. 해지일처럼 미래 정보가 담긴 컬럼은 제거합니다.

예시 2: 다음 달 매출 예측

문제 유형은 회귀 또는 시계열 예측입니다. 월별 추세, 계절성, 프로모션, 공휴일 정보를 피처로 만들 수 있습니다. 매출 분포가 심하게 치우쳐 있으면 로그 변환을 검토하고, 평가는 원래 매출 단위의 MAE/RMSE로도 확인합니다.

예시 3: 고객 유형 나누기

정답 라벨이 없다면 비지도학습의 군집화 문제입니다. 구매 빈도, 최근 구매일, 평균 구매 금액 같은 피처를 만들고 표준화 후 K-Means 등을 적용할 수 있습니다. 결과는 군집별 특징을 사람이 해석할 수 있어야 합니다.

마지막 요약

머신러닝 설계의 핵심은 순서입니다. 문제를 정의하고, 데이터가 그 문제를 답할 수 있는지 확인하고, 단순한 기준 모델을 만든 뒤, 전처리와 피처를 다듬고, 올바른 지표로 평가합니다. 복잡한 모델과 튜닝은 그 다음입니다.

이 문서는 초안입니다. 이후 실제 코드 예제, scikit-learn 파이프라인, 프로젝트별 템플릿을 추가하면 학습용 교재에서 실습용 워크북으로 확장할 수 있습니다.

17. 보강 노트: 놓치기 쉬운 실전 포인트

앞 장까지가 머신러닝 프로젝트의 기본 흐름이라면, 이 장은 실제 작업에서 자주 빠지는 확인 사항입니다. 모델 성능이 생각보다 낮거나, 평가 결과가 믿기 어렵거나, 운영 단계에서 문제가 생길 때 특히 중요합니다.

EDA: 데이터 탐색부터 시작하기

전처리와 모델링 전에 먼저 데이터를 관찰해야 합니다. 컬럼 타입, 결측치 비율, 이상치, 타깃 분포, 변수 간 상관관계를 확인하면 어떤 전처리가 필요한지 훨씬 명확해집니다.

확인 항목	보는 이유
컬럼 타입	숫자, 범주, 날짜, 텍스트를 구분해야 전처리 방식이 정해집니다.
타깃 분포	회귀에서는 치우침, 분류에서는 클래스 불균형을 확인합니다.
결측치와 이상치	삭제, 대체, 변환 중 어떤 처리가 필요한지 판단합니다.
상관관계	강하게 관련된 변수, 중복 정보, 누수 의심 변수를 찾을 수 있습니다.

피처 엔지니어링

피처 엔지니어링은 원본 데이터를 모델이 더 잘 이해할 수 있는 형태로 바꾸는 작업입니다. 많은 프로젝트에서 모델 변경보다 좋은 피처 추가가 더 큰 성능 개선을 만듭니다.

- 날짜에서 요일, 월, 분기, 주말 여부, 공휴일 여부를 만듭니다.
- 구매 데이터에서 최근 구매일, 구매 횟수, 평균 구매 금액 같은 RFM 변수를 만듭니다.
- 총액보다 비율이 중요한 경우 구매 전환율, 할인율, 이탈률 같은 비율 변수를 만듭니다.
- 고객별, 상품별, 지역별 평균·합계·횟수 같은 그룹 통계량을 만들 수 있습니다.

그룹 통계량을 만들 때도 데이터 누수를 조심해야 합니다. 예측 시점 이후의 정보가 평균이나 합계에 섞이면 실제 운영 성능보다 평가 성능이 좋아 보일 수 있습니다.

과적합과 과소적합

상황	의미	대응
Train 성능은 좋고 Test 성능은 나쁨	과적합 가능성	모델 단순화, 정규화, early stopping, 데이터 추가, 피처 정리
Train과 Test 성능이 모두 나쁨	과소적합 가능성	피처 추가, 더 표현력 있는 모델 사용, 학습 부족 확인

분류 Threshold 조정

분류 모델은 보통 확률을 출력하고, 기본적으로 0.5 이상이면 Positive로 판단합니다. 하지만 0.5가 항상 최선은 아닙니다. 미탐을 줄이고 싶으면 threshold를 낮춰 Recall을 높일 수 있고, 오탐을 줄이고 싶으면 threshold를 높여 Precision을 높일 수 있습니다.

예를 들어 사기 거래 탐지에서는 사기를 놓치는 비용이 크다면 threshold를 낮춰 더 많이 잡도록 설계할 수 있습니다. 반대로 마케팅 대상자 선정처럼 잘못 보낸 연락의 비용이 크다면 Precision을 더 중시할 수 있습니다.

모델별 전처리 차이

모델 계열	전처리 포인트
선형/로지스틱 회귀	표준화, 이상치 처리, 다중공선성 확인이 중요합니다.
KNN, SVM, PCA	거리와 스케일에 민감하므로 표준화가 거의 필수에 가깝습니다.
트리 기반 모델	스케일링 영향은 작지만 결측치, 범주형 처리 방식은 모델별로 확인해야 합니다.
신경망	스케일링, 충분한 데이터, regularization, early stopping이 중요합니다.

파이프라인과 재현성

전처리와 모델을 따로 관리하면 학습 때와 예측 때 처리 방식이 달라지는 실수가 생기기 쉽습니다. 가능하면 전처리와 모델을 하나의 파이프라인으로 묶고, 학습 데이터에는 `fit`, 검증·테스트·운영 데이터에는 `transform` 만 적용하는 흐름을 지켜야 합니다.

- random seed를 고정해 실험 결과를 재현할 수 있게 합니다.
- 학습 데이터 버전, 피쳐 목록, 전처리 코드, 모델 파일을 함께 관리합니다.
- 모델 성능뿐 아니라 학습 날짜, 사용한 지표, 주요 하이퍼파라미터도 기록합니다.

윤리, 개인정보, 공정성

사람에게 영향을 주는 모델은 성능만으로 충분하지 않습니다. 개인정보 사용이 적절한지, 특정 집단에 불리하게 작동하지 않는지, 예측 결과를 설명할 수 있는지 확인해야 합니다. 특히 채용, 금융, 의료, 교육처럼 민감한 영역에서는 공정성과 설명 가능성이 설계 단계부터 포함되어야 합니다.